

DBMS COURSE PROJECT - SQL SERVER

Lost & Found Campus System

A relational database that lets students report lost or found items, automatically surfaces likely matches, and routes claims through admin approval.

SUBMISSION

25 July 2026

PRESENTATION

26 July 2026

ENGINE

Microsoft SQL Server

Nabia Sajid - Group Leader

Muhammad Abdullah

Maryam Muazzam

Sathya Narayan

THE PROBLEM

Campus items go missing. Recovery is left to chance.

There is no shared system to report a lost item, register a found one, or verify who it actually belongs to - so most items never make it back.

Today, without a system

A found item sits at a front desk. A student who lost it has no way to know it was ever handed in - reports live in group chats, notice boards, or nowhere at all.

What the database solves

One shared source of truth: every lost and found report, every candidate match, every claim and its approval - all queryable, all auditable.

CORE FEATURES

Five features, one schema

Every feature below maps directly onto a table, a join, a trigger, or a stored procedure - nothing here is UI-only.

Report Lost Item

Student logs title, category, location and date of a lost item.

Report Found Item

Same structure, filed by whoever found it.

Match Similar Items

Engine auto-suggests Lost/Found pairs by category, location, date window.

Claim Requests

Student requests ownership of a found item with proof details.

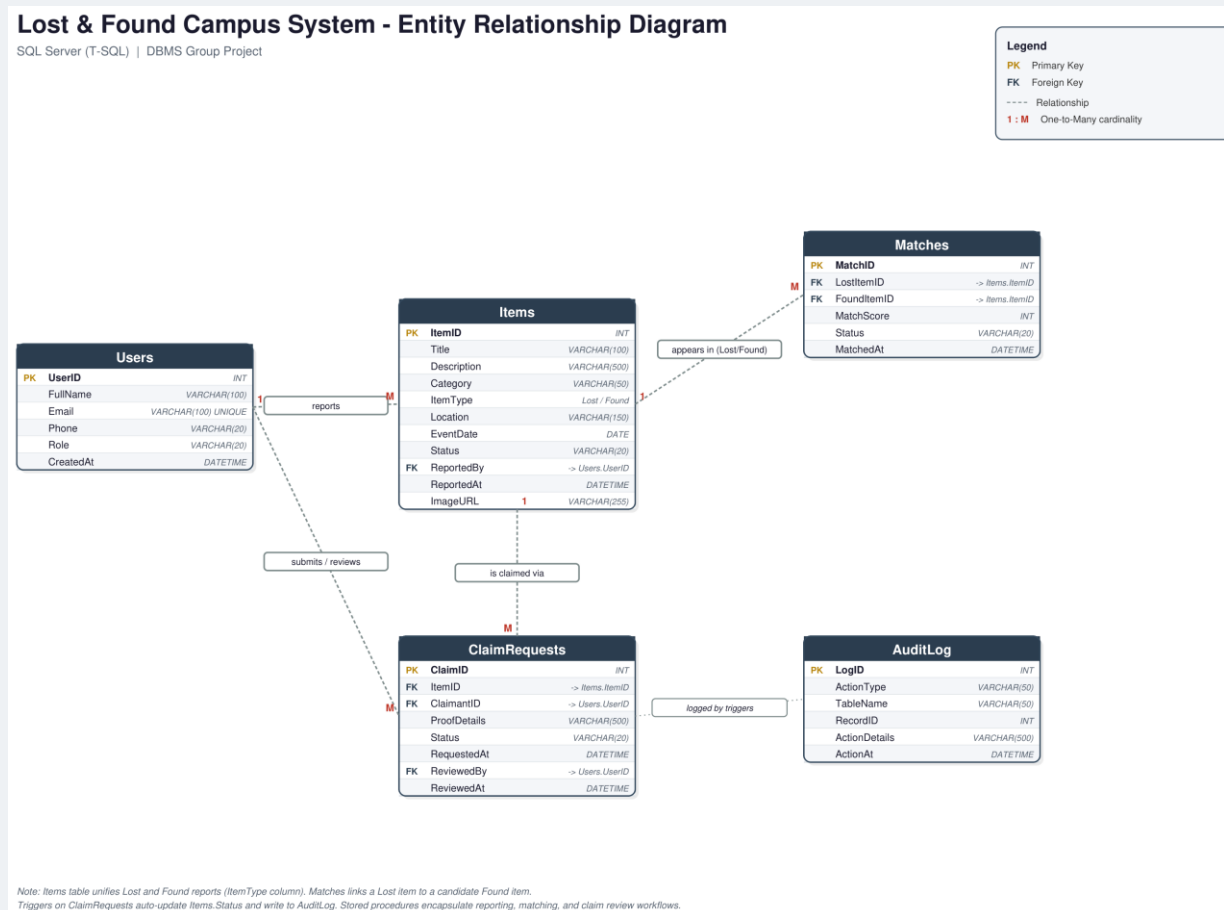
Admin Approval

Admin approves or rejects; approval finalizes the hand-over.

DATA MODEL

Entity-Relationship Diagram

Five tables. One unified Items table for both Lost and Found rows, distinguished by an ItemType column.



SCHEMA

Table by table

Primary keys in gold, foreign keys in teal - every FK enforces referential integrity back to its parent table.

Users	Items	Matches
PK UserID FullName, Email Phone, Role	PK ItemID Title, Category ItemType, Status FK ReportedBy -> Users	PK MatchID FK LostItemID -> Items FK FoundItemID -> Items MatchScore, Status
ClaimRequests	AuditLog	
PK ClaimID FK ItemID -> Items FK ClaimantID -> Users Status, ProofDetails	PK LogID ActionType, TableName ActionDetails, ActionAt	

CONCEPT 1 - JOINS

Lost items with reporter details

Joining Items to Users turns a raw ReportedBy number into a name and email an admin can actually act on.

```
SELECT i.ItemID, i.Title, i.Category, i.Location, i.EventDate,  
       u.FullName AS ReportedBy, u.Email  
FROM Items i  
JOIN Users u ON i.ReportedBy = u.UserID  
WHERE i.ItemType = 'Lost';
```

ITEMID	TITLE	CATEGORY	LOCATION	REPORTED BY	EMAIL
1	Black Wallet	Wallet	Library	Ali Raza	ali.raza@campus.edu
2	iPhone 13	Electronics	Cafeteria	Sara Khan	sara.khan@campus.edu
3	Blue Water Bottle	Bottle	Sports Ground	Bilal Ahmed	bilal.ahmed@campus.edu

Output captured from SQL Server Management Studio

CONCEPT 2 - TRIGGERS

Status updates itself

When an admin approves a claim, trg_ClaimRequest_Approved fires automatically - no application code has to remember to update the item.

```
-- fires AFTER UPDATE on ClaimRequests

UPDATE i SET i.Status = 'Claimed'
FROM Items i JOIN inserted ins ON i.ItemID = ins.ItemID
WHERE ins.Status = 'Approved';

-- auto-reject any other pending claim on the same item
UPDATE cr SET cr.Status = 'Rejected'
FROM ClaimRequests cr JOIN inserted ins ON cr.ItemID = ins.ItemID
WHERE ins.Status = 'Approved' AND cr.ClaimID <> ins.ClaimID;
```

OPEN

->

CLAIMPENDING

->

CLAIMED

VERIFIED IN SSMS

CONCEPT 3 - STORED PROCEDURES

sp_MatchItems - auto-matching

One call scans every open Lost and Found item and inserts every plausible pair into Matches, scored by location.

```
INSERT INTO Matches (LostItemID, FoundItemID, MatchScore, Status)
SELECT l.ItemID, f.ItemID,
       CASE WHEN l.Location = f.Location THEN 80 ELSE 50 END, 'Suggested'
FROM Items l
JOIN Items f ON l.Category = f.Category
              AND l.ItemType = 'Lost' AND f.ItemType = 'Found'
              AND f.EventDate BETWEEN l.EventDate AND DATEADD(DAY, 7, l.EventDate)
WHERE l.Status = 'Open' AND f.Status = 'Open';
```

MATCHID	LOST ITEM	FOUND ITEM	SCORE	STATUS
1	#1 Black Wallet	#4 Wallet Found	80	Suggested
2	#2 iPhone 13	#5 Phone Found	80	Suggested
3	#3 Blue Water Bottle	#6 Water Bottle Found	80	Suggested

VERIFICATION

End-to-end test, traced live

Every row below was captured directly from SSMS while running 06_demo.sql - not simulated.

COMPONENT	RESULT	EVIDENCE
Schema + constraints	PASS	5 tables, all FKs enforced
Join queries	PASS	6/6 returned correct combined data
sp_MatchItems	PASS	3/3 lost items correctly matched
trg_ClaimRequest_Insert	PASS	Item status -> ClaimPending on claim
sp_ReviewClaim + approval trigger	PASS	Item status -> Claimed on approval
AuditLog	PASS	INSERT + UPDATE both logged automatically

THANK YOU

Questions?

Lost & Found Campus System - a working SQL Server database demonstrating joins, triggers, and stored procedures end to end.

SQL Server / T-SQL

5 Tables

6 Joins

3 Triggers

4 Procedures